

## IO\_ADC.h File Reference

---

IO Driver functions for ADC. [More...](#)

```
#include "IO_Driver.h"
```

## Data Structures

---

struct **IO\_ADC\_SAFETY\_CONF**  
Safety configuration for the ADC inputs. [More...](#)

---

## Macros

---

### Types of configurable Analog inputs

Input configuration for configurable ADC inputs. These defines can be used for the `type` parameter of the function `IO_ADC_ChannelInit()`.

```
#define IO_ADC_RATIOMETRIC 0x00U
```

```
#define IO_ADC_CURRENT 0x01U
```

```
#define IO_ADC_RESISTIVE 0x02U
```

```
#define IO_ADC_ABSOLUTE 0x03U
```

---

### Pull up / Pull down configuration for ADC inputs

Configuration of the pull up or pull down resistors on the ADC inputs. These defines can be used for the `pupd` parameter of the function `IO_ADC_ChannelInit()`.

```
#define IO_ADC_NO_PULL 0x00U
```

```
#define IO_ADC_PU_10K 0x02U
```

```
#define IO_ADC_PD_10K 0x01U
```

---

### Range configuration for ADC inputs

Configuration of the ADC input range. These defines can be used for the `range` parameter of the function `IO_ADC_ChannelInit()`.

```
#define IO_ADC_NO_RANGE 0x00U
```

```
#define IO_ADC_RANGE_5V 0x01U
```

```
#define IO_ADC_RANGE_10V 0x02U
```

```
#define IO_ADC_RANGE_32V 0x03U
```

---

## Functions

---

**IO\_ErrorType**

**IO\_ADC\_ChannelInit** (**ubyte1** adc\_channel, **ubyte1** type, **ubyte1** range, **ubyte1** pupd, **ubyte1** sensor\_supply, const **IO\_ADC\_SAFETY\_CONF** \*const safety\_conf)  
Setup one ADC channel. [More...](#)

---

**IO\_ErrorType** **IO\_ADC\_ChannelDeInit** (**ubyte1** adc\_channel)  
Deinitializes one ADC input. [More...](#)

---

**IO\_ErrorType** **IO\_ADC\_Get** (**ubyte1** adc\_channel, **ubyte4** \*const adc\_value, **bool** \*const fresh)  
Returns the value of the given ADC channel. [More...](#)

---

**float4** **IO\_ADC\_BoardTempFloat** (**ubyte4** raw\_value)  
Calculates the board temperature in degree Celsius. [More...](#)

---

**sbyte2** **IO\_ADC\_BoardTempSbyte** (**ubyte4** raw\_value)  
Calculates the board temperature in degree Celsius. [More...](#)

---

**IO\_ErrorType** **IO\_ADC\_ResetProtection** (**ubyte1** adc\_channel, **ubyte1** \*const reset\_cnt)  
Reset the FET protection for an ADC channel. [More...](#)

---

## Detailed Description

---

IO Driver functions for ADC.

Contains all service functions for the ADC. There are three groups of ADC inputs:

- 3 mode ADC:
  - 5V ADC inputs.
  - Up to 8 ADC inputs for 0-5V measurement ([IO\\_ADC\\_00..IO\\_ADC\\_07](#)).
  - Can be configured as resistive, current, absolute or ratiometric input. An additional reference channel or a sensor supply measurement (for the ratiometric case) will be configured and serves to correct the ADC signal.
  - Range: 0-5V
- 2 mode 10V ADC:
  - Software configurable ADC inputs.
  - Up to 8 ADC inputs for 0-10V measurement ([IO\\_ADC\\_08..IO\\_ADC\\_15](#)).
  - Can be configured as current, absolute or ratiometric input. An additional reference channel or a sensor supply measurement (for the ratiometric case) will be configured and serves to correct the ADC signal.
  - Range: can be configured to 0-5V or 0-10V for absolute and ratiometric measurement.
- 2 mode 32V ADC:
  - Software configurable ADC inputs.
  - Up to 8 ADC inputs for 0-32V measurement ([IO\\_ADC\\_16..IO\\_ADC\\_23](#)).
  - Can be configured as current, absolute or ratiometric input. An additional reference channel or a sensor supply measurement (for the ratiometric case) will be configured and serves to correct the ADC signal.
  - Range: can be configured to 0-5V or 0-32V for absolute and ratiometric measurement.
- Normal ADC:

- Various ADC inputs for retrieving onboard voltages (UBat, Sensor-Supply, Board-Temperature, ...)

### Note

The ADC channels will be sampled every 2ms. After this time a new measurement is available and will be flagged as fresh via the parameter `fresh` of `IO_ADC_Get()`.

### Attention

For `IO_ADC_RATIOMETRIC` mode the following configuration options are recommended:

- 3 mode ADC: (`IO_ADC_00..IO_ADC_07`) with `IO_ADC_SENSOR_SUPPLY_0` or `IO_ADC_SENSOR_SUPPLY_1`.
- 3 mode ADC: (`IO_ADC_00..IO_ADC_07`) with `IO_ADC_SENSOR_SUPPLY_2` (sensor supply voltage setting 5V ... 10V).
- 2 mode 10V ADC: (`IO_ADC_08..IO_ADC_15`) configured in `IO_ADC_RANGE_5V` with `IO_ADC_SENSOR_SUPPLY_0` or `IO_ADC_SENSOR_SUPPLY_1`.
- 2 mode 10V ADC: (`IO_ADC_08..IO_ADC_15`) configured in `IO_ADC_RANGE_5V` range with `IO_ADC_SENSOR_SUPPLY_2` (sensor supply voltage setting 5V ... 10V)
- 2 mode 10V ADC: (`IO_ADC_08..IO_ADC_15`) configured in `IO_ADC_RANGE_10V` range with `IO_ADC_SENSOR_SUPPLY_2` (sensor supply voltage setting 6V ... 10V).

The precision of the remaining configuration options in `IO_ADC_RATIOMETRIC` mode for reading a ratiometric sensor connected to one of the sensor supplies is mainly limited by the fact that the related stages are designed for input voltages up to 32V or 55V, therefore the precision of those inputs is only slightly improved by selecting the ratiometric mode. **The configuration options not listed above are NOT recommended.**

ADC-API Usage:

- [Examples for ADC API functions](#)

## ADC input protection

Each ADC input that is configured for current measurement mode is individually protected against overcurrent. Whenever two consecutive samples above 27234uA are measured, the input protection is enabled latest within 22ms.

When entering the protection state, the ADC input has to remain in this state for at least 1s. After this wait time the ADC input can be reenabled via function `IO_ADC_ResetProtection()`. Note that the number of reenabling operations for a single ADC input is limited to 10. Refer to function `IO_ADC_ResetProtection()` for more information on how to reenable an ADC input.

### Attention

ADC input protection is only applicable to ADC channels `IO_ADC_00 .. IO_ADC_23` when configured for current measurement mode.

# ADC Code Examples

Examples for using the ADC API

## Example for ADC initialization

```
// 3 mode ADC:
IO_ADC_ChannelInit(IO_ADC_00,
                   IO_ADC_RATIOMETRIC,           // ratiometric configuration
                   IO_ADC_NO_RANGE,
                   IO_ADC_NO_PULL,
                   IO_SENSOR_SUPPLY_0,           // sensor supply 0 is used
                   NULL);                        // not safety critical

// 2 mode 10V ADC:
IO_ADC_ChannelInit(IO_ADC_08,
                   IO_ADC_ABSOLUTE,              // absolute configuration
                   IO_ADC_RANGE_10V,             // 10V configuration
                   IO_ADC_NO_PULL,
                   IO_PIN_NONE,
                   NULL);                        // not safety critical

// Normal ADC:
IO_ADC_ChannelInit(IO_ADC_UBAT,
                   IO_ADC_ABSOLUTE,
                   IO_ADC_RANGE_32V,
                   IO_ADC_NO_PULL,
                   IO_PIN_NONE,
                   NULL);                        // not safety critical
```

## Example for ADC task function call

This function call is identical for every type of ADC inputs.

```
ubyte4 adc_val_20;
bool adc_fresh_20;
IO_ErrorType rc;

rc = IO_ADC_Get(IO_ADC_20,
                &adc_val_20,
                &adc_fresh_20);
```

## Macro Definition Documentation

```
#define IO_ADC_ABSOLUTE 0x03U
```

Absolute voltage measurement

use this configuration to measure an absolute voltage signal

Task function returns voltage in [mV]

```
#define IO_ADC_CURRENT 0x01U
```

Current loop configuration

use this configuration if the connected sensor delivers a current signal (4..25mA sensors)

Task function returns current in [uA]

```
#define IO_ADC_NO_PULL 0x00U
```

fixed pull resistor

```
#define IO_ADC_NO_RANGE 0x00U
```

ADC range not configurable

```
#define IO_ADC_PD_10K 0x01U
```

10 kOhm pull down

```
#define IO_ADC_PU_10K 0x02U
```

10 kOhm pull up

```
#define IO_ADC_RANGE_10V 0x02U
```

ADC range 0V .. 10V

```
#define IO_ADC_RANGE_32V 0x03U
```

ADC range 0V .. 32V

```
#define IO_ADC_RANGE_5V 0x01U
```

ADC range 0V .. 5V

```
#define IO_ADC_RATIOMETRIC 0x00U
```

Ratiometric configuration

use this configuration if the connected sensor is supplied by one of the sensor supplies (`IO_SENSOR_SUPPLY_0` or `IO_SENSOR_SUPPLY_1` or `IO_SENSOR_SUPPLY_2`) and delivers a voltage signal.

Task function returns voltage in [mV]

```
#define IO_ADC_RESISTIVE 0x02U
```

Resistive configuration

use this configuration if the sensor value shall be determined by measuring its resistance

Task function returns resistance in [Ohm]

## Function Documentation

---

```
float4 IO_ADC_BoardTempFloat ( ubyte4 raw_value )
```

Calculates the board temperature in degree Celsius.

The function converts the raw ADC value (retrieved from `IO_ADC_Get()`) to a temperature in degree Celsius and returns it as a float value.

### Parameters

**raw\_value** raw adc board temperature returned from the `IO_ADC_Get()` function

### Returns

the board temperature in degree Celsius ( -63.00 .. 152.50 degree C )

### Remarks

Usage:

```
IO_ADC_Get( IO_ADC_BOARD_TEMP,
            &raw_value,
            &fresh );

temp = IO_ADC_BoardTempFloat( raw_value );
```

**sbyte2 IO\_ADC\_BoardTempSbyte ( ubyte4 raw\_value )**

Calculates the board temperature in degree Celsius.

The function converts the raw ADC value (retrieved from `IO_ADC_Get()`) to a temperature in degree Celsius and returns it as a signed value.

**Parameters**

**raw\_value** raw adc board temperature returned from the `IO_ADC_Get()` function

**Returns**

the board temperature in degree Celsius ( -63 .. 152 degree C )

**Remarks**

Usage:

```
IO_ADC_Get( IO_ADC_BOARD_TEMP,
            &raw_value,
            &fresh );

temp = IO_ADC_BoardTempSbyte( raw_value );
```

**IO\_ErrorType IO\_ADC\_ChannelDeInit ( ubyte1 adc\_channel )**

Deinitializes one ADC input.

deinitializes the given ADC channel, allows reconfiguration by `IO_ADC_ChannelInit()`

#### Parameters

**adc\_channel** ADC channel, one of:

- `IO_ADC_00 .. IO_ADC_07`
- `IO_ADC_08 .. IO_ADC_15`
- `IO_ADC_16 .. IO_ADC_23`
- `IO_ADC_24 .. IO_ADC_35`
- `IO_ADC_36 .. IO_ADC_43`
- `IO_ADC_44 .. IO_ADC_51`
- `IO_ADC_52 .. IO_ADC_59`
- `IO_ADC_SENSOR_SUPPLY_0`
- `IO_ADC_SENSOR_SUPPLY_1`
- `IO_ADC_SENSOR_SUPPLY_2`
- `IO_ADC_K15`
- `IO_ADC_WAKE_UP`
- `IO_ADC_UBAT`
- `IO_ADC_SAFETY_SW_0`
- `IO_ADC_SAFETY_SW_1`
- `IO_ADC_SAFETY_SW_2`
- `IO_ADC_BOARD_TEMP`

#### Returns

**IO\_ErrorType**

#### Return values

|                                    |                                                            |
|------------------------------------|------------------------------------------------------------|
| <b>IO_E_OK</b>                     | everything fine                                            |
| <b>IO_E_INVALID_CHANNEL_ID</b>     | the given channel id does not exist                        |
| <b>IO_E_CH_CAPABILITY</b>          | the given channel is not an ADC channel                    |
| <b>IO_E_CHANNEL_NOT_CONFIGURED</b> | the given channel is not configured                        |
| <b>IO_E_DRIVER_NOT_INITIALIZED</b> | the common driver init function has not been called before |
| <b>IO_E_UNKNOWN</b>                | an unknown error occurred                                  |



```

IO_ErrorType IO_ADC_ChannelInit ( ubyte1                adc_channel,
                                   ubyte1                type,
                                   ubyte1                range,
                                   ubyte1                pupd,
                                   ubyte1                sensor_supply,
                                   const IO_ADC_SAFETY_CONF *const safety_conf
                                   )

```

Setup one ADC channel.

### Parameters

**adc\_channel**    ADC channel, one of:

- `IO_ADC_00 .. IO_ADC_07`
- `IO_ADC_08 .. IO_ADC_15`
- `IO_ADC_16 .. IO_ADC_23`
- `IO_ADC_24 .. IO_ADC_35`
- `IO_ADC_36 .. IO_ADC_43`
- `IO_ADC_44 .. IO_ADC_51`
- `IO_ADC_52 .. IO_ADC_59`
- `IO_ADC_SENSOR_SUPPLY_0`
- `IO_ADC_SENSOR_SUPPLY_1`
- `IO_ADC_SENSOR_SUPPLY_2`
- `IO_ADC_K15`
- `IO_ADC_WAKE_UP`
- `IO_ADC_UBAT`
- `IO_ADC_SAFETY_SW_0`
- `IO_ADC_SAFETY_SW_1`
- `IO_ADC_SAFETY_SW_2`
- `IO_ADC_BOARD_TEMP`

**type**    Type of input:

- `IO_ADC_RATIOMETRIC`: voltage input on sensor supply
- `IO_ADC_CURRENT`: 0-25mA input
- `IO_ADC_RESISTIVE`: 0-100000Ohm input
- `IO_ADC_ABSOLUTE`: normal voltage input

**range**    Range of input:

- `IO_ADC_NO_RANGE`: voltage input with fixed range
- `IO_ADC_RANGE_5V`: voltage input 0-5000mV
- `IO_ADC_RANGE_10V`: voltage input 0-10200mV

- `IO_ADC_RANGE_32V`: voltage input 0-32000mV

**pupd**

Pull up/down configuration:

- `IO_ADC_NO_PULL`: fixed pull resistor
- `IO_ADC_PU_10K`: pull up 10 kOhm
- `IO_ADC_PD_10K`: pull down 10 kOhm

**sensor\_supply** Sensor supply:

- `IO_SENSOR_SUPPLY_0`: 5V
- `IO_SENSOR_SUPPLY_1`: 5V
- `IO_SENSOR_SUPPLY_2`: variable
- `IO_PIN_NONE`: no sensor supply is used

`[in]` **safety\_conf**

Relevant safety configurations for the checker modules. The following ADC channels can be configured safety relevant:

- `IO_ADC_00 .. IO_ADC_07`
- `IO_ADC_08 .. IO_ADC_15`
- `IO_ADC_16 .. IO_ADC_23`

**Returns**`IO_ErrorType`**Return values**

|                                              |                                                                                                |
|----------------------------------------------|------------------------------------------------------------------------------------------------|
| <code>IO_E_OK</code>                         | everything fine                                                                                |
| <code>IO_E_INVALID_CHANNEL_ID</code>         | the given channel id does not exist or is not available on the used ECU variant                |
| <code>IO_E_CH_CAPABILITY</code>              | the given channel is not an ADC channel or the used ECU variant does not support this function |
| <code>IO_E_INVALID_PARAMETER</code>          | parameter is out of range                                                                      |
| <code>IO_E_CHANNEL_BUSY</code>               | the ADC input channel is currently used by another function                                    |
| <code>IO_E_DRIVER_NOT_INITIALIZED</code>     | the common driver init function has not been called before                                     |
| <code>IO_E_INVALID_SAFETY_CONFIG</code>      | an invalid safety configuration has been passed                                                |
| <code>IO_E_DRV_SAFETY_CONF_NOT_CONFIG</code> | global safety configuration is missing                                                         |
| <code>IO_E_DRV_SAFETY_CYCLE_RUNNING</code>   | the init function was called after the task begin function                                     |
| <code>IO_E_UNKNOWN</code>                    | an unknown error occurred                                                                      |

## Remarks

- The supported features depend on the selected channel:
  - `IO_ADC_00 .. IO_ADC_07`:
    - type: `IO_ADC_RATIOMETRIC`, `IO_ADC_CURRENT`, `IO_ADC_RESISTIVE` or `IO_ADC_ABSOLUTE`
    - sensor\_supply: `IO_SENSOR_SUPPLY_0`, `IO_SENSOR_SUPPLY_1` or `IO_SENSOR_SUPPLY_2`
    - safety\_conf: Safety configuration
  - `IO_ADC_08 .. IO_ADC_15`:
    - type: `IO_ADC_RATIOMETRIC`, `IO_ADC_CURRENT` or `IO_ADC_ABSOLUTE`
    - range: `IO_ADC_RANGE_5V` or `IO_ADC_RANGE_10V` (supported for type `IO_ADC_ABSOLUTE` and `IO_ADC_RATIOMETRIC`)
    - sensor\_supply: `IO_SENSOR_SUPPLY_0`, `IO_SENSOR_SUPPLY_1` or `IO_SENSOR_SUPPLY_2`
    - safety\_conf: Safety configuration
  - `IO_ADC_16 .. IO_ADC_23`:
    - type: `IO_ADC_RATIOMETRIC`, `IO_ADC_CURRENT` or `IO_ADC_ABSOLUTE`
    - range: `IO_ADC_RANGE_5V` or `IO_ADC_RANGE_32V` (supported for type `IO_ADC_ABSOLUTE` and `IO_ADC_RATIOMETRIC`)
    - sensor\_supply: `IO_SENSOR_SUPPLY_0`, `IO_SENSOR_SUPPLY_1` or `IO_SENSOR_SUPPLY_2`
    - safety\_conf: Safety configuration
  - `IO_ADC_24 .. IO_ADC_35`
    - type: `IO_ADC_RATIOMETRIC` or `IO_ADC_ABSOLUTE`
    - sensor\_supply: `IO_SENSOR_SUPPLY_0`, `IO_SENSOR_SUPPLY_1` or `IO_SENSOR_SUPPLY_2` (supported for type `IO_ADC_RATIOMETRIC`)
    - pupd: `IO_ADC_PU_10K` or `IO_ADC_PD_10K`
  - `IO_ADC_36 .. IO_ADC_43`
    - type: `IO_ADC_RATIOMETRIC` or `IO_ADC_ABSOLUTE`
    - sensor\_supply: `IO_SENSOR_SUPPLY_0`, `IO_SENSOR_SUPPLY_1` or `IO_SENSOR_SUPPLY_2` (supported for type `IO_ADC_RATIOMETRIC`)
    - pupd: `IO_ADC_PU_10K` or `IO_ADC_PD_10K`
  - `IO_ADC_44 .. IO_ADC_51`
    - type: `IO_ADC_RATIOMETRIC` or `IO_ADC_ABSOLUTE`
    - sensor\_supply: `IO_SENSOR_SUPPLY_0`, `IO_SENSOR_SUPPLY_1` or `IO_SENSOR_SUPPLY_2` (supported for type `IO_ADC_RATIOMETRIC`)

- `IO_ADC_52 .. IO_ADC_59`
  - type: `IO_ADC_RATIOMETRIC` or `IO_ADC_ABSOLUTE`
  - sensor\_supply: `IO_SENSOR_SUPPLY_0`, `IO_SENSOR_SUPPLY_1` or `IO_SENSOR_SUPPLY_2` (supported for type `IO_ADC_RATIOMETRIC`)
  - pupd: `IO_ADC_PU_10K` or `IO_ADC_PD_10K`
- `IO_ADC_SENSOR_SUPPLY_0`:
  - type: `IO_ADC_ABSOLUTE`
- `IO_ADC_SENSOR_SUPPLY_1`:
  - type: `IO_ADC_ABSOLUTE`
- `IO_ADC_SENSOR_SUPPLY_2`:
  - type: `IO_ADC_ABSOLUTE`
- `IO_ADC_K15`
  - type: `IO_ADC_ABSOLUTE`
- `IO_ADC_WAKE_UP`
  - type: `IO_ADC_ABSOLUTE`
- `IO_ADC_UBAT`
  - type: `IO_ADC_ABSOLUTE`
- `IO_ADC_SAFETY_SW_0`
  - type: `IO_ADC_ABSOLUTE`
- `IO_ADC_SAFETY_SW_1`
  - type: `IO_ADC_ABSOLUTE`
- `IO_ADC_SAFETY_SW_2`
  - type: `IO_ADC_ABSOLUTE`
- `IO_ADC_BOARD_TEMP`
  - type: `IO_ADC_ABSOLUTE`

- If a channel does not support a function, the value of the associated parameter will be ignored.

### Attention

For safety critical configuration (`safety_conf != NULL`) following restrictions additionally apply:

- Resistive measurement type `IO_ADC_RESISTIVE` is not valid
- Variable sensor supply `IO_SENSOR_SUPPLY_2` is not valid

### Note

If a channel is configured as safety critical and a sensor supply was configured, also the sensor supply voltage will be checked by the diagnostic modules.



```
IO_ErrorType IO_ADC_Get ( ubyte1      adc_channel,
                          ubyte4 *const adc_value,
                          bool *const  fresh
                        )
```

Returns the value of the given ADC channel.

### Parameters

**adc\_channel** ADC channel, one of:

- `IO_ADC_00 .. IO_ADC_07`
- `IO_ADC_08 .. IO_ADC_15`
- `IO_ADC_16 .. IO_ADC_23`
- `IO_ADC_24 .. IO_ADC_35`
- `IO_ADC_36 .. IO_ADC_43`
- `IO_ADC_44 .. IO_ADC_51`
- `IO_ADC_52 .. IO_ADC_59`
- `IO_ADC_SENSOR_SUPPLY_0`
- `IO_ADC_SENSOR_SUPPLY_1`
- `IO_ADC_SENSOR_SUPPLY_2`
- `IO_ADC_K15`
- `IO_ADC_WAKE_UP`
- `IO_ADC_UBAT`
- `IO_ADC_SAFETY_SW_0`
- `IO_ADC_SAFETY_SW_1`
- `IO_ADC_SAFETY_SW_2`
- `IO_ADC_BOARD_TEMP`

[out] **adc\_value** ADC value, the range depends on the input group and its configuration (type parameter of `IO_ADC_Channellnit()`):

- `IO_ADC_00 .. IO_ADC_07`
  - `IO_ADC_RATIOMETRIC`: 0..5000 (0V..5.000V)
  - `IO_ADC_CURRENT`: 0..25000 (0mA..25.000mA)
  - `IO_ADC_RESISTIVE`: 0..100000 (0Ohm..100000Ohm)
  - `IO_ADC_ABSOLUTE`: 0..5000 (0V..5.000V)
- `IO_ADC_08 .. IO_ADC_15`
  - `IO_ADC_RATIOMETRIC`: 0..5000 (0V..5.000V)
  - `IO_ADC_CURRENT`: 0..25000 (0mA..25.000mA)
  - `IO_ADC_ABSOLUTE`: 0..10200 (0V..10.200V)
- `IO_ADC_16 .. IO_ADC_23`
  - `IO_ADC_RATIOMETRIC`: 0..5000 (0V..5.000V)
  - `IO_ADC_CURRENT`: 0..25000 (0mA..25.000mA)

- `IO_ADC_ABSOLUTE`: 0..32000 (0V..32.000V)
- `IO_ADC_24 .. IO_ADC_35`
  - `IO_ADC_RATIOMETRIC`: 0..32000 (0V..32.000V)
  - `IO_ADC_ABSOLUTE`: 0..32000 (0V..32.000V)
- `IO_ADC_36 .. IO_ADC_43`
  - `IO_ADC_RATIOMETRIC`: 0..32000 (0V..32.000V)
  - `IO_ADC_ABSOLUTE`: 0..32000 (0V..32.000V)
- `IO_ADC_44 .. IO_ADC_51`
  - `IO_ADC_RATIOMETRIC`: 0..32000 (0V..32.000V)
  - `IO_ADC_ABSOLUTE`: 0..32000 (0V..32.000V)
- `IO_ADC_52 .. IO_ADC_59`
  - `IO_ADC_RATIOMETRIC`: 0..32000 (0V..32.000V)
  - `IO_ADC_ABSOLUTE`: 0..32000 (0V..32.000V)
- `IO_ADC_SENSOR_SUPPLY_0`
  - `IO_ADC_ABSOLUTE`: 0..5254 (0V..5.254V)
- `IO_ADC_SENSOR_SUPPLY_1`
  - `IO_ADC_ABSOLUTE`: 0..5254 (0V..5.254V)
- `IO_ADC_SENSOR_SUPPLY_2`
  - `IO_ADC_ABSOLUTE`: 0..10560 (0V..10.560V)
- `IO_ADC_K15`
  - `IO_ADC_ABSOLUTE`: 0..55000 (0V..55.000V)
- `IO_ADC_WAKE_UP`
  - `IO_ADC_ABSOLUTE`: 0..55000 (0V..55.000V)
- `IO_ADC_UBAT`
  - `IO_ADC_ABSOLUTE`: 0..55000 (0V..55.000V)
- `IO_ADC_SAFETY_SW_0`
  - `IO_ADC_ABSOLUTE`: 0..55000 (0V..55.000V)
- `IO_ADC_SAFETY_SW_1`
  - `IO_ADC_ABSOLUTE`: 0..55000 (0V..55.000V)
- `IO_ADC_SAFETY_SW_2`
  - `IO_ADC_ABSOLUTE`: 0..55000 (0V..55.000V)
- `IO_ADC_BOARD_TEMP`
  - `IO_ADC_ABSOLUTE`: 3700..25250 (-63.00..152.50 degree C)

[out] **fresh**

Status of the ADC value

- `TRUE`: ADC value is fresh (channel has been converted)
- `FALSE`: ADC value is old (channel has not been converted)

## Returns

`IO_ErrorType`

**Return values**

|                                          |                                                                                                                                                                      |
|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>IO_E_OK</code>                     | everything fine                                                                                                                                                      |
| <code>IO_E_INVALID_CHANNEL_ID</code>     | the given channel id does not exist                                                                                                                                  |
| <code>IO_E_CH_CAPABILITY</code>          | the given channel is not an ADC channel                                                                                                                              |
| <code>IO_E_STARTUP</code>                | the input is in the startup phase                                                                                                                                    |
| <code>IO_E_FET_PROT_ACTIVE</code>        | the input FET protection is active and has deactivated the internal switch. It can be reset with <code>IO_ADC_ResetProtection()</code> after the wait time is passed |
| <code>IO_E_FET_PROT_REENABLE</code>      | the input FET protection is ready to be reset with <code>IO_ADC_ResetProtection()</code>                                                                             |
| <code>IO_E_FET_PROT_PERMANENT</code>     | the input FET is protected permanently because it was already reset more than 10 times                                                                               |
| <code>IO_E_REFERENCE</code>              | A reference voltage (sensor supply or internal reference) is out of range.                                                                                           |
| <code>IO_E_NULL_POINTER</code>           | a NULL pointer has been passed                                                                                                                                       |
| <code>IO_E_CHANNEL_NOT_CONFIGURED</code> | the given channel is not configured                                                                                                                                  |
| <code>IO_E_DRIVER_NOT_INITIALIZED</code> | the common driver init function has not been called before                                                                                                           |
| <code>IO_E_UNKNOWN</code>                | an unknown error occurred                                                                                                                                            |

**Remarks**

- If there is no new measurement value available (for example the function `IO_ADC_Get()` gets called more frequently than the AD sampling) the flag `fresh` will be set to `FALSE`.
- The temperature value in degree C must be recalculated in the following manner, since the value has an offset of 10000 and is multiplied with 100:  

$$\text{degree\_value} = ((\text{float4}) \text{adc\_value} - 10000) / 100$$



```
IO_ErrorType IO_ADC_ResetProtection ( ubyte1      adc_channel,
                                     ubyte1 *const reset_cnt
                                     )
```

Reset the FET protection for an ADC channel.

### Parameters

**adc\_channel** ADC channel:

- **IO\_ADC\_00 .. IO\_ADC\_07:**
  - configured type: **IO\_ADC\_CURRENT**
- **IO\_ADC\_08 .. IO\_ADC\_15**
  - configured type: **IO\_ADC\_CURRENT**
- **IO\_ADC\_16 .. IO\_ADC\_23**
  - configured type: **IO\_ADC\_CURRENT**

[out] **reset\_cnt** Protection reset counter. Indicates how often the application already reset the protection.

### Returns

**IO\_ErrorType**

### Return values

|                                    |                                                                                         |
|------------------------------------|-----------------------------------------------------------------------------------------|
| <b>IO_E_OK</b>                     | everything fine                                                                         |
| <b>IO_E_INVALID_CHANNEL_ID</b>     | the given channel id does not exist                                                     |
| <b>IO_E_FET_PROT_NOT_ACTIVE</b>    | no input FET protection is active                                                       |
| <b>IO_E_FET_PROT_PERMANENT</b>     | the input FET is protected permanently because it was already reset more than 10 times  |
| <b>IO_E_FET_PROT_WAIT</b>          | the input FET protection can not be reset, as the wait time of 1s is not already passed |
| <b>IO_E_CHANNEL_NOT_CONFIGURED</b> | the given channel is not configured                                                     |
| <b>IO_E_CH_CAPABILITY</b>          | the given channel has no FET protection                                                 |
| <b>IO_E_UNKNOWN</b>                | an unknown error occurred                                                               |

### Attention

The protection can be reset 10 times, afterwards the input will remain permanently protected

### Note

- The input can only enter a protection state if configured in current mode
- After entering the input protection, a certain time has to pass before the input protection can be reset:

- 1s for `IO_ADC_00 .. IO_ADC_07`
- 1s for `IO_ADC_08 .. IO_ADC_15`
- 1s for `IO_ADC_16 .. IO_ADC_23`

- This function will close the internal switch to enable the input back again. After calling this function new measurement values can be read with `IO_ADC_Get()`

#### Remarks

If the parameter `reset_cnt` is `NULL`, the parameter is ignored. The parameter `reset_cnt` returns the number of resets already performed.

---

Generated on Wed Jan 11 2017 13:55:49 for HY-TTC 500 I/O Driver by  1.8.6